CashSouk P2P Lending Platform — AWS Deployment Architecture

Stack: Next.js (Landing + 3 Portals) + Express API + PostgreSQL on **AWS ECS Fargate**, **RDS**, **S3 + CloudFront**, **Cognito**.
Region: `ap-southeast-5` (Malaysia).
Purpose: Complete AWS deployment guide for CashSouk P2P lending platform with automated CI/CD.

---

1) High-Level Architecture

CloudFront (+WAF) → ALB → 5 ECS Fargate Services:

- `cashsouk-landing` (Next.js SSR) — landing page
- `cashsouk-investor` (Next.js SSR) — investor portal
- `cashsouk-issuer` (Next.js SSR) — issuer portal
- `cashsouk-admin` (Next.js SSR) — admin dashboard
- `cashsouk-api` (Express) — REST API

Infrastructure:

- **RDS PostgreSQL (Multi-AZ)** behind **RDS Proxy** (private subnets)
- **S3 (private, SSE-KMS, versioning)** for file uploads
- **CloudFront** for static assets + global CDN
- **AWS Cognito User Pool** for authentication
- **Secrets Manager / SSM Parameter Store** for configuration
- **VPC:** 3 AZs; ALB in public subnets; ECS tasks + RDS in private subnets
- **VPC Endpoints** for S3/SSM/Logs (avoid NAT costs)
- **CloudWatch** for logs, metrics, alarms

> ***Data Residency:*** *All compute + data in* `ap-southeast-5` *(Malaysia).*

---

2) Repository Structure (Current Monorepo)

```
cashsouk/
├── apps/
│   ├── landing/          # Next.js landing page (port 3000)
│   ├── investor/         # Next.js investor portal (port 3002)
│   ├── issuer/           # Next.js issuer portal (port 3001)
│   ├── admin/            # Next.js admin dashboard (port 3003)
│   └── api/              # Express API (port 4000)
├── packages/
│   ├── ui/               # shadcn/ui components
│   ├── styles/           # Tailwind config & design tokens
│   ├── types/            # Shared TypeScript types
│   ├── config/           # API client & configuration
│   ├── icons/            # Icon library
│   └── testing/          # Test utilities
├── docker/
│   ├── landing.Dockerfile
│   ├── portal-investor.Dockerfile
│   ├── portal-issuer.Dockerfile
│   ├── portal-admin.Dockerfile
│   └── api.Dockerfile
├── infra/
│   ├── ecs/              # ECS task definitions
│   └── README.md         # AWS setup guide
├── scripts/
```

```
│   ├── migrate.sh        # Prisma migration runner
│   ├── setup-aws.sh      # AWS infrastructure helper
│   ├── build-all.sh      # Local Docker builds
│   └── ecs-update.sh     # ECS service updater
├── .github/
│   └── workflows/
│       ├── deploy.yml    # Production deployment
│       └── test.yml      # PR testing
├── docker-compose.yml        # Development environment
├── docker-compose.prod.yml   # Production testing
├── Makefile              # Development tasks
└── turbo.json            # Turborepo configuration
```

---

3) AWS Resource Naming Conventions

ECR Repositories:

- `cashsouk-landing`
- `cashsouk-investor`
- `cashsouk-issuer`
- `cashsouk-admin`
- `cashsouk-api`

ECS:

- **Cluster:** `cashsouk-prod`
- **Services:** `cashsouk-landing`, `cashsouk-investor`, `cashsouk-issuer`, `cashsouk-admin`, `cashsouk-api`
- **Task Definitions:** (same names as services)

RDS:

- **Instance:** `cashsouk-prod-postgres`
- **Proxy:** `cashsouk-prod-rds-proxy`

S3:

- `cashsouk-prod-uploads-<account-id>-ap-southeast-5`
- `cashsouk-prod-logs-<account-id>-ap-southeast-5`

CloudFront:

- Distribution: `cashsouk-prod-cdn`

Cognito:

- User Pool: `cashsouk-prod-users`

WAF:

- ACL: `cashsouk-prod-waf`

---

4) Docker Standards (Production)

Next.js Portals (landing, investor, issuer, admin)

All Next.js apps already configured with:

- **Node 20** runtime
- **Multi-stage builds** (builder + runner)
- **Standalone output** (`output: "standalone"` in next.config.js)

- **Internal PORT=3000** (standardized across all portals)
- **Health checks** built-in

Existing Dockerfiles in `docker/`:

- `landing.Dockerfile`
- `portal-investor.Dockerfile`
- `portal-issuer.Dockerfile`
- `portal-admin.Dockerfile`

Express API

Existing `docker/api.Dockerfile`:

- Node 20 Alpine
- Multi-stage build
- Health check at `/healthz`
- Internal PORT=3000 (maps to 4000 externally in dev)
- Prisma Client generated during build

---

5) ALB Routing Strategy

Host-Based Routing (single ALB, HTTPS listener on port 443):

Host	Target Group	Service
`cashsouk.com`	`tg-landing`	cashsouk-landing
`investor.cashsouk.com`	`tg-investor`	cashsouk-investor
`issuer.cashsouk.com`	`tg-issuer`	cashsouk-issuer
`admin.cashsouk.com`	`tg-admin`	cashsouk-admin
`api.cashsouk.com`	`tg-api`	cashsouk-api

Health Checks:

- Portals: `GET /` (status 200)
- API: `GET /healthz` (status 200)

TLS:

- ACM wildcard certificate: `*.cashsouk.com`
- Primary certificate: `cashsouk.com`

---

6) Environment Variables

Development (.env.local)

All Portals (landing/investor/issuer/admin):

```
NODE_ENV=development
NEXT_PUBLIC_API_URL=http://localhost:4000
NEXT_PUBLIC_COGNITO_DOMAIN=
NEXT_PUBLIC_COGNITO_CLIENT_ID=
NEXT_PUBLIC_COGNITO_REGION=ap-southeast-5
```

API:

```
NODE_ENV=development
PORT=4000
DATABASE_URL=postgresql://postgres:password@localhost:5432/cashsouk_dev
AWS_REGION=ap-southeast-5
S3_BUCKET=
COGNITO_USER_POOL_ID=
COGNITO_APP_CLIENT_ID=
ALLOWED_ORIGINS=http://localhost:3000,http://localhost:3001,http://localhost:3002,http://localhost:3003
```

Production (.env.prod for local testing)

Portals:

```
NODE_ENV=production
NEXT_PUBLIC_API_URL=https://api.cashsouk.com
NEXT_PUBLIC_COGNITO_DOMAIN=https://cashsouk-prod.auth.ap-southeast-5.amazoncognito.com
NEXT_PUBLIC_COGNITO_REGION=ap-southeast-5
NEXT_PUBLIC_CLOUDFRONT_URL=https://assets.cashsouk.com
```

API:

```
NODE_ENV=production
PORT=3000
AWS_REGION=ap-southeast-5
S3_PREFIX=uploads/
ALLOWED_ORIGINS=https://cashsouk.com,https://investor.cashsouk.com,https://issuer.cashsouk.com,https://
```

AWS Production (SSM Parameter Store)

Environment variables injected via ECS task definitions from:

- `/cashsouk/prod/api/database-url` (Secrets Manager)
- `/cashsouk/prod/api/rds-proxy-endpoint`
- `/cashsouk/prod/cognito/user-pool-id`
- `/cashsouk/prod/cognito/client-id`
- `/cashsouk/prod/s3/bucket-name`
- `/cashsouk/prod/cloudfront/domain`

> ***Never commit .env files.*** *Use* `.env.local` *and* `.env.prod` *(gitignored).*

---

7) CI/CD Pipeline (GitHub Actions → AWS)

Authentication

GitHub OIDC (no static AWS keys):

- Configure GitHub as identity provider in AWS IAM
- Create role: `github-actions-deployer`
- Trust policy allows GitHub repo to assume role
- Permissions: ECR push, ECS update, SSM read, Secrets Manager read

Workflow Triggers

`.github/workflows/deploy.yml`:

- Trigger: push to `main` branch

- Conditional paths (deploy only changed services)

`.github/workflows/test.yml`:

- Trigger: pull requests to `main`
- No deployment, only testing

Deployment Flow

```
Push to main
  ↓
[1] Run Tests
  ├─ Lint (ESLint)
  ├─ Type check (TypeScript)
  ├─ Unit tests (Jest)
  └─ E2E tests (Playwright)
  ↓
[2] Build & Push Images
  ├─ Login to ECR (OIDC)
  ├─ Build changed services
  ├─ Tag with git SHA + latest
  └─ Push to ECR
  ↓
[3] Run Migrations (if API changed)
  ├─ ECS run-task (migrations task def)
  ├─ Execute: prisma migrate deploy
  └─ Wait for completion
  ↓
[4] Deploy Services
  ├─ Update task definitions
  ├─ Update ECS services
  ├─ Wait for stability
  └─ Health check verification
  ↓
[5] Notify
  └─ Slack/Email on success/failure
```

Monorepo Optimization

Turborepo + pnpm:

- Shared package caching
- Parallel builds where possible
- Affected services detection via path filters

Build Commands:

```
pnpm install --frozen-lockfile
pnpm -w typecheck
pnpm -w lint
pnpm -w test
pnpm -w build
```

---

8) Database Migrations (Prisma)

Strategy: Never run migrations from web containers.

Options:

A) Pre-deploy Migration Task (Recommended):

- Separate ECS task definition: `task-def-migrations.json`
- GitHub Actions runs: `aws ecs run-task`
- Uses API image with only migration command
- Waits for success before service updates

B) Manual Approval:

- Protected workflow with approval step
- Engineer reviews migrations
- Triggers migration task manually

Migration Task:

```
# In migration container
cd apps/api
npx prisma migrate deploy
```

Task Definition:

- Same network (private subnet)
- Same task role (RDS access)
- Command override: `["sh", "-c", "cd apps/api && npx prisma migrate deploy"]`
- No health checks (one-shot task)

---

9) S3 File Upload Strategy

Bucket Configuration:

- Private (no public access)
- SSE-KMS encryption
- Versioning enabled
- Lifecycle policy (archive after 90 days)

Upload Flow:

1. Client requests presigned PUT URL from API
2. API validates user authorization
3. API generates presigned URL (15-min expiry)
4. Client uploads directly to S3
5. Client notifies API of upload completion
6. API stores metadata in database

API Routes:

```
POST /api/v1/files/upload-url
  → { uploadUrl, key, expiresIn }

POST /api/v1/files/complete
  → { fileId, url }
```

CloudFront Serving:

- Public assets via OAC (Origin Access Control)
- Private files via presigned CloudFront URLs (API-generated)

---

10) Security Checklist

- [x] WAF on CloudFront (AWS Managed Rules)
- [x] ALB in public subnets, ECS + RDS in private
- [x] Security groups: least-privilege
- [x] IAM roles: task-specific, no wildcards
- [x] RDS Proxy for connection pooling
- [x] S3 bucket policies: deny public access
- [x] VPC endpoints (S3, SSM, Logs) — no NAT
- [x] TLS everywhere (ACM certificates)
- [x] HSTS headers on CloudFront
- [x] Secrets in Secrets Manager (not env vars)
- [x] CloudWatch alarms: 5XX, latency, CPU, DB
- [x] Backup automation (RDS snapshots daily)
- [x] Cost anomaly detection enabled

---

11) Observability

CloudWatch Logs:

- Log group per service: `/ecs/cashsouk-{service}`
- Retention: 30 days
- JSON structured logging (Pino for API)

Metrics:

- ECS task CPU/Memory utilization
- ALB request count, latency, 5XX rate
- RDS connections, CPU, storage

Alarms:

- API 5XX rate > 1% (5 min)
- Portal 5XX rate > 0.5% (5 min)
- RDS CPU > 80% (10 min)
- RDS storage < 20% free (immediate)
- ECS task unhealthy (immediate)

Dashboards:

- System overview (all services)
- Per-service drill-down
- Database performance
- Cost tracking

---

12) Deployment Environments

Environment	Purpose	Trigger	Database
Local Dev	Development	Manual	Local Postgres
Local Prod	Production testing	Manual	Local Postgres
AWS Staging	Pre-production	PR merge	RDS (small)

AWS Prod	Production	Push to main	RDS (Multi-AZ)

Future: Add staging environment with separate AWS resources.

---

13) Initial AWS Setup (One-Time)

Before first deployment, manually create:

1. **VPC & Networking:**
   - VPC (10.0.0.0/16)
   - 3 public subnets (ALB)
   - 3 private subnets (ECS + RDS)
   - Internet Gateway
   - NAT Gateway (or VPC endpoints)
   - Route tables
2. **Load Balancer:**
   - Application Load Balancer
   - Target groups (5 total)
   - Listener rules (host-based)
   - Security groups
3. **ECS:**
   - Cluster: `cashsouk-prod`
   - Task execution role
   - Task roles (per service)
4. **RDS:**
   - PostgreSQL 15 (Multi-AZ)
   - Subnet group (private subnets)
   - Security group (ECS only)
   - RDS Proxy
   - Initial database & user
5. **ECR:**
   - 5 repositories (landing, investor, issuer, admin, api)
   - Lifecycle policies (keep 10 images)
6. **S3:**
   - Upload bucket
   - Log bucket
   - Bucket policies
7. **CloudFront:**
   - Distribution
   - OAC for S3
   - Cache policies
   - WAF association
8. **Cognito:**
   - User pool
   - App clients (web + mobile)
   - Domain

- Identity pool (optional)

9. **IAM:**
   - GitHub OIDC provider
   - Deployment role
   - Task execution roles
   - Task roles
   - **API Task Role Permissions:**
     - `cognito-idp:AdminUserGlobalSignOut` (optional, for server-side logout)
     - RDS access (via RDS Proxy)
     - S3 read/write (for file uploads)
     - Secrets Manager read (for environment variables)
     - **Note:** If `AdminUserGlobalSignOut` permission is not granted, logout will still work but will only clear local tokens/cookies. The Cognito session will remain active until it expires naturally.

10. **SSM/Secrets:**
    - Parameter paths
    - Initial values

11. **Route 53:**
    - Hosted zone
    - A records (ALB)
    - CNAME (CloudFront)

12. **ACM:**
    - Certificate request
    - DNS validation

See `infra/README.md` for detailed steps.

---

14) Cost Optimization

Estimated Monthly Cost (ap-southeast-5):

- ECS Fargate (5 services, 0.5vCPU, 1GB): ~$50
- RDS PostgreSQL (db.t4g.small, Multi-AZ): ~$60
- ALB: ~$25
- NAT Gateway (or VPC endpoints): ~$30
- S3 + CloudFront: ~$10
- Data transfer: ~$10
- **Total: ~$185/month**

Optimization:

- Use VPC endpoints instead of NAT ($30 → $7)
- Auto-scaling (scale to zero off-hours for dev)
- Fargate Spot for non-critical tasks
- S3 Intelligent Tiering
- CloudFront caching (reduce origin requests)

---

15) Disaster Recovery

RTO: 1 hour
RPO: 5 minutes

Backups:

- RDS automated snapshots (daily)
- Manual snapshots before major changes
- S3 versioning + replication (optional)

Recovery Procedures:

1. Deploy from last known good commit
2. Restore RDS from snapshot
3. Verify data integrity
4. Switch traffic (ALB target groups)

Quarterly DR Drills:

- Test RDS restore
- Verify backup integrity
- Document recovery time

---

16) Quick Reference Commands

```
# Local development
make dev                # Start docker-compose
make build              # Build all images
make test               # Run tests
make migrate            # Run migrations

# Production testing
make deploy-local       # Test production build locally

# AWS deployment
git push origin main    # Triggers automatic deployment

# Manual operations
./scripts/setup-aws.sh              # Initial AWS setup
./scripts/migrate.sh production     # Run migrations in AWS
./scripts/build-all.sh              # Build all Docker images
```

---

17) Monitoring & Alerts

Slack Notifications:

- Deployment success/failure
- High error rates
- Resource exhaustion
- Cost anomalies

PagerDuty (optional):

- Critical production alerts
- On-call rotation

Log Aggregation:

- CloudWatch Logs Insights
- Custom queries for debugging
- Saved queries for common issues

---

18) Compliance & Auditing

For Malaysian regulatory compliance:

- All data at rest encrypted (RDS, S3)
- Data residency in ap-southeast-5
- CloudTrail enabled (API audit logs)
- VPC Flow Logs (network audit)
- Access logging (ALB, S3)
- Regular security scans
- Quarterly compliance reviews

Last Updated: 2025-01-24
Maintained by: CashSouk Engineering Team
Questions: See `infra/README.md` or team documentation